

GEM-garfield-simulation

Installation Guide

Last verified: 2026-09-26

Contents

1	Overview	1
1.1	Why installation order matters	2
1.2	Versions verified to work together	2
2	ROOT	2
3	Gmsh	3
3.1	Gmsh CLI (optional)	3
3.2	Gmsh Python API (required)	3
4	Elmer FEM	3
5	Garfield++	4
5.1	Rebuilding after a ROOT upgrade	4
5.2	A CMake gotcha worth knowing about in advance	4
6	Python environment	5
6.1	The <code>vtk</code> version pin (required)	5
6.2	Versions this project is actually tested against	5
7	Building this project's C++ macros	5
8	Where paths are set (no dependence on shell startup files)	6
8.1	Optional: speeding up shell startup on a networked home directory	6
9	Verifying the full stack end-to-end	6
10	Summary of known compatibility pitfalls	7

1 Overview

This document is a detailed, step-by-step installation guide for the software stack that **GEM-garfield-simulation** depends on: **Gmsh** (geometry/mesh generation), **Elmer FEM** (electrostatic field solve), **Garfield++/Magboltz** (microscopic electron avalanche simulation), **ROOT** (I/O and, indirectly, the ABI that Garfield++ must be built against), and the **Python** environment used for analysis and visualization.

`README.md` in the repository root gives the minimum a returning user needs (a version table and where paths are set); this document is the fuller reference for someone setting this stack up for

the first time, or rebuilding a piece of it after an upgrade.

1.1 Why installation order matters

The one constraint that drives most of the pain in this stack is: **Garfield++ must be built against the exact same ROOT installation (same version, same compiler ABI) that everything else in the pipeline uses.** ROOT’s dictionary-generation ABI changes between versions, so a Garfield++ build linked against ROOT 6.32 will fail to link against a ROOT 6.40 executable with errors like:

```
undefined reference to ‘ROOT::TGenericClassInfo::TGenericClassInfo(...)’  
undefined reference to ‘TGeoBBox::ComputeNormal(...)’
```

There is no way around this other than keeping one consistent ROOT version and rebuilding Garfield++ against it whenever that version changes. The recommended order is therefore:

1. Install ROOT first, and settle on the version you intend to keep.
2. Install Gmsh and Elmer (independent of ROOT/Garfield++).
3. Build Garfield++ against that same ROOT installation.
4. Set up the Python environment (independent, but see Section 6 for a real package-conflict gotcha).
5. Build this repository’s own C++ macros against the same ROOT + Garfield++ pair.

1.2 Versions verified to work together

The following combination is what this project is actually built and tested against (2026-09-26). Later point releases of any one component are not guaranteed to be compatible with the others without re-testing.

Tool	Version
Gmsh (CLI)	4.13.1
Gmsh Python API (pip install gmsh)	4.15.2
Elmer FEM	v9.0
Garfield++	package version string “0.3” (built against ROOT below)
ROOT	6.40.04
CMake	3.31.8
gcc	11.5.0 (Red Hat 11.5.0-14)

2 ROOT

Install ROOT first, since Garfield++’s build depends on it. Follow ROOT’s own build instructions (https://root.cern/install/build_from_source/) for your platform; in short, for a from-source build:

```
git clone --branch v6-40-04 --depth 1 https://github.com/root-project/root.git  
root_src  
mkdir root_build && cd root_build  
cmake -DCMAKE_INSTALL_PREFIX=$HOME/local/root/6.40.04 ../root_src  
cmake --build . -j"$(nproc)"
```

```
cmake --install .
```

After installing, source ROOT’s environment script before using it:

```
source $HOME/local/root/6.40.04/bin/thisroot.sh
root-config --version # sanity check
```

This project does *not* rely on `thisroot.sh` being sourced globally (e.g. from `.bashrc`) – each script/build step sets the paths it needs explicitly (see Section 8), so that the shell startup used for day-to-day work is not slowed down by always loading a full ROOT environment. Set it up however is convenient for your own workflow; just be consistent about which ROOT installation every other tool below is pointed at.

3 Gmsh

Two independent things are needed: the Gmsh CLI (only used incidentally; this project drives Gmsh almost entirely through its Python API) and the Gmsh Python module.

3.1 Gmsh CLI (optional)

Download a prebuilt binary from <https://gmsh.info/#Download> and extract it, e.g.:

```
mkdir -p ~/local
cd ~/local
wget https://gmsh.info/bin/Linux/gmsh-4.13.1-Linux64.tgz
tar xf gmsh-4.13.1-Linux64.tgz
~/local/gmsh-4.13.1-Linux64/bin/gmsh --version
```

3.2 Gmsh Python API (required)

```
pip install gmsh
python3 -c "import gmsh; print(gmsh.__file__)"
```

The pip package’s own bundled version (4.15.2 here) does not need to exactly match the standalone CLI’s version above; they are independent installs and this project only actually calls the Python API at runtime.

Import-order gotcha: in some conda/pip environments, importing `gmsh` before `matplotlib` in the same Python process causes

```
ImportError: /usr/lib64/libstdc++.so.6: version ‘CXXABI_1.3.15’ not found
```

Gmsh’s native extension can pull in the system’s older `libstdc++.so.6` first; once that specific shared object is loaded process-wide, matplotlib’s compiled extension (which needs a newer `libstdc++` ABI) can no longer load the version it actually needs. The fix is purely at the Python-import level: **always import matplotlib (or any module that imports it) before import gmsh** in any script that uses both. This project’s own scripts already follow this convention; keep it if you add a new one that touches both.

4 Elmer FEM

Build from source (<http://www.elmerfem.org/blog/binaries/> also has prebuilt binaries for some platforms, but a from-source build is more portable across clusters):

```
git clone https://github.com/ElmerCSC/elmerfem.git elmer_src
mkdir elmer_build && cd elmer_build
cmake -DCMAKE_INSTALL_PREFIX=$HOME/elmer \
      -DWITH_MPI=OFF \
      ../elmer_src
cmake --build . -j"$(nproc)"
cmake --install .
```

`-DWITH_MPI=OFF` is a deliberate simplification here: this project's meshes are solved as a single linear system per run (not domain-decomposed), so MPI parallelism inside `ElmerSolver` is not needed, and a non-MPI build avoids an entire class of shared-library version-mismatch problems (e.g. a system `libmpi_usempif08.so` mismatch) that a shared/central Elmer install on a cluster can otherwise run into. If your own environment already has a working MPI-enabled Elmer install, that works too; just confirm `ElmerSolver` actually runs (some shared HPC installs turn out to be broken in ways that only surface at runtime, not at link/install time).

Verify with:

```
export ELMER_HOME=$HOME/elmer
export PATH="$ELMER_HOME/bin:$PATH"
export LD_LIBRARY_PATH="$ELMER_HOME/lib:${LD_LIBRARY_PATH:-}"
ElmerSolver -version
```

5 Garfield++

Build from the official source (<https://gitlab.cern.ch/garfield/garfieldpp>), pointed explicitly at the ROOT installation from Section 2:

```
git clone https://gitlab.cern.ch/garfield/garfieldpp.git garfieldpp_src
mkdir -p garfieldpp_src/build && cd garfieldpp_src/build

export ROOTSYS=$HOME/local/root/6.40.04
export PATH="$ROOTSYS/bin:$PATH"
export LD_LIBRARY_PATH="$ROOTSYS/lib:$LD_LIBRARY_PATH"

cmake -DROOT_DIR="$ROOTSYS/cmake" \
      -DCMAKE_INSTALL_PREFIX=$HOME/local/garfield \
      ..
cmake --build . -j"$(nproc)"
cmake --install .
```

5.1 Rebuilding after a ROOT upgrade

If ROOT is ever upgraded, Garfield++ **must** be rebuilt against the new installation (rerun the same `cmake/build/install` sequence above with `ROOTSYS` pointed at the new ROOT) – reusing the old `libGarfield.so` against a newer ROOT is exactly the ABI mismatch described in Section 1.1. If the Garfield++ install is shared with other projects, confirm with anyone else relying on it before rebuilding it in place.

5.2 A CMake gotcha worth knowing about in advance

On at least one system, a fresh `cmake ..` for a project consuming Garfield++ (including this repository's own `macros/CMakeLists.txt`) can silently resolve `ROOT_DIR` back to a stale/wrong ROOT installation, even after `CMAKE_PREFIX_PATH` is set correctly. This happens

because Garfield’s own `GarfieldConfig.cmake` internally calls `find_dependency(ROOT 6.0 COMPONENTS Geom Gdml)`, which can win the resolution race depending on what `CMAKE_PREFIX_PATH` was inherited from the environment. If `cmake ..`’s output reports the wrong ROOT under a line like `Found Vdt: ...`, force it explicitly:

```
cmake -DROOT_DIR=$HOME/local/root/6.40.04/cmake ..
```

or add `set(ROOT_DIR "." CACHE PATH "" FORCE)` to the consuming project’s own `CMakeLists.txt` right after its `CMAKE_PREFIX_PATH` is set (this repository’s `macros/CMakeLists.txt` already does this – see that file’s own comments).

6 Python environment

Any Python 3 environment (conda, venv, or a plain `pip install --user`) works; this project does not depend on a specific environment manager. Install:

```
pip install gmsh numpy matplotlib uproot pyvista plotly
pip install "vtk==9.3.1"
```

6.1 The vtk version pin (required)

Installing `pyvista` alone pulls in whatever the latest `vtk` wheel is. As of 2026-09, that is `9.7.x`, which is **incompatible** with `trame_vtk` (the library `pyvista`’s HTML export path uses) and fails with:

```
TypeError: unhashable type 'VTKAOSArray_vtkFloatArray'
```

while serializing a scene for HTML export (used by `visualization/plot_triple_gem.py`). Explicitly pinning `vtk==9.3.1` (after installing `pyvista`, so the pin takes effect) resolves this. If a future `trame_vtk` release fixes the incompatibility, this pin can likely be relaxed – but treat that as a deliberate, tested decision, not an assumption.

6.2 Versions this project is actually tested against

Package	Version
numpy	2.4.4
matplotlib	3.10.9
uproot	5.7.4
pyvista	0.49.0
plotly	7.1.0
vtk	9.3.1 (pinned, see above)
gmsh (Python)	4.15.2

Note: `uproot` (not `PyROOT`) is what this project’s Python code uses to read ROOT files. `PyROOT` is deliberately not used here, so the Python environment’s own version does not need to match ROOT’s Python binding at all – one less version constraint to track.

7 Building this project’s C++ macros

Once ROOT, Garfield++, and Elmer are installed and consistent with each other:

```
cd GEM-garfield-simulation/macros
mkdir -p build && cd build
cmake ..
cmake --build . -j"${nproc}"
```

macros/CMakeLists.txt points CMAKE_PREFIX_PATH at \$HOME/local/garfield and \$HOME/local/root/6.40.04 by default – edit those two paths at the top of that file if your own installation locations differ from this project’s own development machine.

If `cmake ..` reports the wrong ROOT (see Section 3.3), pass `-DROOT_DIR=...` explicitly as shown there.

8 Where paths are set (no dependence on shell startup files)

This project deliberately does **not** rely on `.bashrc` (or similar) exporting `ROOTSYS`, `ELMER_HOME`, or `GARFIELD_HOME` globally. Each entry point sets what it needs explicitly:

- `elmer/run_field_solve.sh` exports `ELMER_HOME/PATH/LD_LIBRARY_PATH` itself before invoking `ElmerGrid/ElmerSolver`.
- `macros/CMakeLists.txt` sets `CMAKE_PREFIX_PATH` (and, if needed, `ROOT_DIR`) itself at configure time.
- Python scripts read ROOT files via `uproot` (pure Python), so they need no ROOT environment at all.

If you set up your own equivalents of these paths differently from the defaults baked into this repository, the two files above are the only places that need editing – there is no third, hidden place any of this is duplicated.

8.1 Optional: speeding up shell startup on a networked home directory

If your home directory is on a networked filesystem (e.g. GPFS/NFS) and you notice a large conda environment or ROOT installation making the *first python/root* invocation of a session slow (tens of seconds, due to many small-file stat/open calls over the network), packing the environment into a single squashfs image and mounting it read-only under a node-local `/tmp` path is one effective fix. This project’s own development machine uses exactly this approach (see `~/local/envfs_README.md` outside this repository for that machine’s specific implementation) – it is a pure performance optimization, entirely optional, and orthogonal to everything else in this guide: batch (LSF) jobs are unaffected by it either way, since they always see the real, unmounted paths.

9 Verifying the full stack end-to-end

Once everything above is installed, confirm the whole pipeline actually runs, from geometry generation through to Python analysis, using the `triple_gem_field` model (this exact command sequence is the one verified end-to-end in `docs/reference.md` §2):

```
cd GEM-garfield-simulation

cd geometry && python3 build_triple_gem_field_mesh.py && cd ..
cd elmer && bash run_field_solve.sh triple_gem_field && cd ..

cd macros/build
```

```
./export_avalanche_trajectories ../../results/mesh/triple_gem_field \  
  ../../resources/ar_ch4_90_10.gas 5 -0.2029 0.8405 0.4235 0.021 \  
  0.03637306695894642 0.1 0.0005 ../../results/root  
cd ../../  
  
python3 visualization/plot_triple_gem.py triple_gem_field  
python3 geometry/analyze_plane_crossings.py \  
  results/root/triple_gem_field_avalanche.root
```

A successful run produces `results/root/triple_gem_field_avalanche.root`, `results/html/triple_gem_field_overview.html`, and prints a funnel/summary from the last command with no Python traceback. See `docs/reference.md` for what each pipeline stage's CLI arguments mean (including how to compute them for a different mesh/model) and how to run the full 7×7 production configuration.

10 Summary of known compatibility pitfalls

- **Garfield++ / ROOT ABI:** must be built against the exact same ROOT version; rebuild Garfield++ after any ROOT upgrade (Section 3.1).
- **CMake resolving the wrong ROOT_DIR:** force it explicitly if a fresh `cmake ..` picks the wrong one (Section 3.3).
- **gmsh vs matplotlib import order:** always import matplotlib-touching modules before gmsh in the same process (Section 3, Gmsh).
- **vtk version:** pin to 9.3.1; the latest wheel breaks pyvista's HTML export (Section 6.1).
- **Elmer with MPI on a shared/HPC install:** confirm `ElmerSolver` actually runs at runtime, not just that it built – a broken shared MPI runtime can pass CMake/link steps and still fail with a shared-library error the first time it is actually invoked (Section 4).

For gotchas encountered building and running the pipeline itself (as opposed to installing the underlying tools), see `docs/pipeline_gotchas.md` in this repository.